

I – Langage local, linéaire et automate de Glushkov

1. Construire l'automate de Glushkov reconnaissant $(ab \mid b)^*b$.

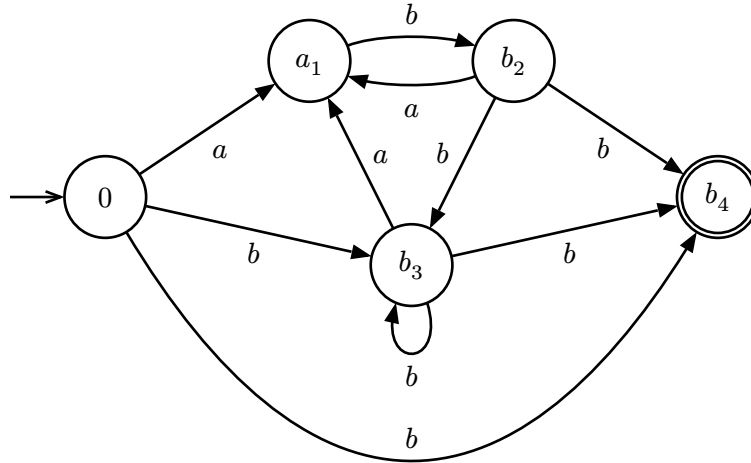
On linéarise l'expression en $(a_1b_2 \mid b_3)^*b_4$. On obtient

$$P = \{a_1, b_3, b_4\},$$

$$S = \{b_4\}$$

et

$$F = \{a_1b_2, b_2a_1, b_2b_3, b_2b_4, b_3a_1, b_3b_3, b_3b_4\}.$$



2. Montrer que si L_1 et L_2 sont locaux alors $L_1 \cap L_2$ est local (même si les alphabets ne sont pas disjoints, contrairement à l'union et à la concaténation vues en cours).

Posons $L = L_1 \cap L_2$ et travaillons sur l'alphabet Σ obtenu, si nécessaire, en réunissant les deux alphabets. Puisque tout mot de L appartient à L_1 et à L_2 , on a

$$P(L) \subseteq P(L_1) \cap P(L_2), \quad S(L) \subseteq S(L_1) \cap S(L_2),$$

$$F(L) \subseteq F(L_1) \cap F(L_2).$$

Soit $u = u_1 \dots u_n \in \Sigma^*$ avec $n \geq 1$. Supposons que $u_1 \in P(L)$, $u_n \in S(L)$ et que tous les facteurs $u_k u_{k+1}$ de longueur deux de u appartiennent à $F(L)$.

Les inclusions précédentes donnent, pour chaque $i \in \{1, 2\}$:

$$u_1 \in P(L_i), \quad u_n \in S(L_i), \quad \forall k \in \{1, \dots, n-1\}, u_k u_{k+1} \in F(L_i).$$

Comme L_i est local, ces conditions entraînent $u \in L_i$. Ainsi $u \in L_1$ et $u \in L_2$, donc $u \in L$.

Enfin, $\varepsilon \in L$ si et seulement si $\varepsilon \in L_1$ et $\varepsilon \in L_2$. Le langage $L = L_1 \cap L_2$ est donc local.

3. Montrer qu'il existe un nombre fini de langages locaux sur un alphabet fixé.

Tout langage local est de la forme

$$(P\Sigma^* \cap \Sigma^*S) \setminus \Sigma^*N\Sigma^*$$

où $P \subseteq \Sigma$, $S \subseteq \Sigma$, $N \subseteq \Sigma \times \Sigma$, en rajoutant éventuellement ε . Si $n = |\Sigma|$, il y a au plus 2^n choix pour P , 2^n pour S et 2^{n^2} pour N , car $|\Sigma \times \Sigma| = n^2$. Il y a donc au plus $2^n \times 2^n \times 2^{n^2}$ valeurs différentes pour l'ensemble $(P\Sigma^* \cap \Sigma^*S) \setminus \Sigma^*N\Sigma^*$, et au plus 2^{n^2+2n+1} langages locaux en tenant compte du choix d'ajouter ε .

4. Soient $\mathcal{L}_{\text{reg}}$, $\mathcal{L}_{\text{loc}}$, $\mathcal{L}_{\text{lin}}$ les ensembles des langages réguliers, locaux et linéaires (c'est-à-dire décrits par une expression régulière linéaire). Quelles sont les inclusions entre ces trois ensembles ? Ces inclusions sont-elles strictes ?

D'après le cours, tout langage décrit par une expression régulière linéaire est local. Tout langage local est reconnaissable, donc régulier par le théorème de Kleene. Ainsi

$$\mathcal{L}_{\text{lin}} \subseteq \mathcal{L}_{\text{loc}} \subseteq \mathcal{L}_{\text{reg}}.$$

Les deux inclusions sont strictes dès que l'alphabet contient une lettre a .

Le langage $L = \{a^n \mid n \geq 1\}$ est local : les premières et dernières lettres sont a , le seul facteur autorisé de longueur deux est aa , et ε est exclu. Il n'est pas linéaire. En effet, dans une expression linéaire produisant aa , l'unique occurrence de a doit être répétée par une étoile. Considérons l'étoile la plus extérieure contenant cette occurrence. Dans cette production, tout ce qui lui est extérieur produit le mot vide, puisqu'il ne contient aucune autre occurrence de a . En prenant zéro répétition, on produirait donc aussi ε , ce qui est impossible pour L .

Le langage dénoté par $(aa)^*$ est régulier mais pas local : puisque aa appartient au langage, ses ensembles de premières et dernières lettres contiennent a . La condition locale accepterait alors aussi le mot a (qui n'a aucun facteur de longueur deux), alors que sa longueur est impaire. Sur l'alphabet vide, les trois familles coïncident avec $\{\emptyset, \{\varepsilon\}\}$.

II – Sujet 0 CCP CCP · 2022 · MPI · oral

1. Rappeler la définition d'un langage régulier.

Sur un alphabet Σ , les langages réguliers forment la plus petite famille contenant les langages finis et stable par union, concaténation et étoile de Kleene.

2. Les langages suivants sont-ils réguliers ? Justifier.

- (a) $L_1 = \{a^n ba^m \mid n, m \in \mathbb{N}\}$ (c) $L_3 = \{a^n ba^m \mid n, m \in \mathbb{N}, n > m\}$
 (b) $L_2 = \{a^n ba^m \mid n, m \in \mathbb{N}, n \leq m\}$ (d) $L_4 = \{a^n ba^m \mid n, m \in \mathbb{N}, n + m \equiv 0 \pmod{2}\}$

(a) L_1 est régulier : il est dénoté par a^*ba^* .

(b) Supposons L_2 régulier. Par le théorème de Kleene, il est reconnaissable par un automate à $n \geq 1$ états. Soit $u = a^n ba^n \in L_2$; on a $|u| = 2n + 1 \geq n$. Soit $u = xyz$ la décomposition donnée par le lemme de l'étoile : $|xy| \leq n$, $y \neq \varepsilon$ et $xy^*z \subseteq L_2$. Comme les n premières lettres de u sont des a , on a $y = a^r$ pour un entier $r \geq 1$. Pour $k = 2$, on obtient

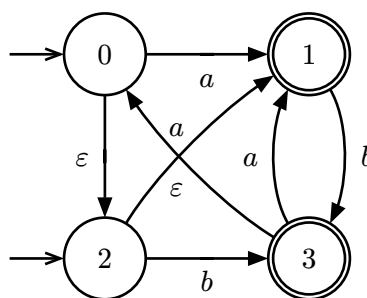
$$xy^2z = a^{n+r}ba^n \notin L_2,$$

puisque $n + r > n$. C'est absurde : L_2 n'est pas reconnaissable, donc n'est pas régulier.

(c) On a $L_2 = L_1 \setminus L_3$. Si L_3 était régulier, la stabilité des langages réguliers par complément et intersection rendrait L_2 régulier, puisque L_1 l'est. Donc L_3 n'est pas régulier.

(d) La somme $n + m$ est paire si et seulement si n et m ont la même parité. Ainsi L_4 est régulier, dénoté par $(aa)^*b(aa)^* \mid a(aa)^*ba(aa)^*$.

3. On considère l'automate non déterministe suivant :

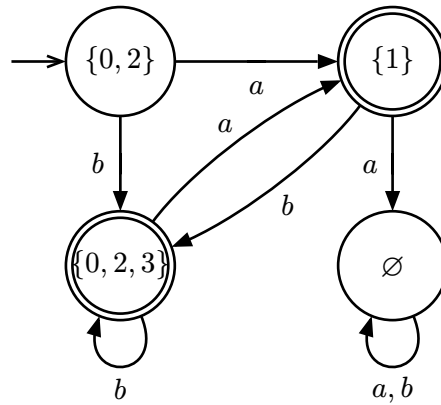


(a) Déterminiser cet automate.

(b) Construire une expression régulière dénotant le langage reconnu par cet automate.

(c) Décrire simplement avec des mots le langage reconnu par cet automate.

(a) Avant la première lecture et après chaque lettre lue, on ajoute tous les états accessibles par zéro, une ou plusieurs ε -transitions, qui se franchissent sans lire de lettre.



(b) Une expression régulière est

$$a \mid (b \mid ab)(b \mid ab)^*(\varepsilon \mid a).$$

Depuis $\{0, 2\}$ ou $\{0, 2, 3\}$, la lecture de b ou de ab mène à $\{0, 2, 3\}$. Après au moins un de ces blocs, on peut s'arrêter dans $\{0, 2, 3\}$ ou lire un dernier a pour atteindre $\{1\}$. Le mot a est accepté directement depuis $\{0, 2\}$. Ce sont tous les chemins acceptants : depuis $\{1\}$, seule la lettre b permet d'éviter le puits $\emptyset$.

(c) Le langage reconnu est l'ensemble des mots non vides sur $\{a, b\}$ ne contenant pas deux a consécutifs. En effet, après un a , on se trouve dans $\{1\}$; un autre a mène au puits. Toute lecture sans facteur aa reste dans $\{1\}$ ou $\{0, 2, 3\}$ dès la première lettre, et est donc acceptée. Le mot vide est rejeté puisque $\{0, 2\}$ n'est pas final.

III – Stabilité des langages réguliers

Pour chacune des propositions suivantes, expliquer pourquoi elle est vraie ou donner un contre-exemple :

1. Si L est régulier et $L \subseteq L'$ alors L' est régulier.

| Faux avec $L = \emptyset$ et $L' = \{a^n b^n \mid n \in \mathbb{N}\}$ sur $\Sigma = \{a, b\}$.

2. Si L' est régulier et $L \subseteq L'$ alors L est régulier.

| Faux avec $L = \{a^n b^n \mid n \in \mathbb{N}\}$ et $L' = \Sigma^*$ sur $\Sigma = \{a, b\}$.

3. Si L est régulier alors L^* est régulier.

| Vrai par définition des langages réguliers.

4. Si L^* est régulier alors L est régulier.

| Faux avec $L = \{a^n b^n \mid n \in \mathbb{N}\} \cup \{\varepsilon, a, b\}$ sur $\Sigma = \{a, b\}$. L^* est alors Σ^* , qui est régulier, mais L ne l'est pas.

5. Si L est régulier sur un alphabet Σ alors $\Sigma^* \setminus L$ (complémentaire de L) est régulier.

| Vrai comme démontré dans le cours sur les automates.

6. Une union finie de langages réguliers est régulière ($L_1, \dots, L_n$ réguliers $\Rightarrow \bigcup_{k=1}^n L_k$ régulier).

| Vrai par récurrence sur n et la stabilité des langages réguliers par union de deux langages réguliers.

7. Une union dénombrable de langages réguliers est régulière ($L_1, \dots, L_n, \dots$ réguliers $\Rightarrow \bigcup_{k=1}^{\infty} L_k$ régulier).

| Faux avec $L_n = \{a^n b^n\}$ pour $n \geq 1$.

8. Une intersection finie de langages réguliers est régulière ($L_1, \dots, L_n$ réguliers $\Rightarrow \bigcap_{k=1}^n L_k$ régulier).

| Vrai par récurrence sur n , en utilisant la stabilité des langages réguliers par intersection de deux langages réguliers, vue dans le cours sur les automates.

9. Une intersection dénombrable de langages réguliers est régulière ($L_1, \dots, L_n, \dots$ réguliers $\Rightarrow \bigcap_{k=1}^{\infty} L_k$ régulier).

| Faux avec $L_n = \Sigma^* \setminus \{a^n b^n\}$ pour $n \geq 1$, sur $\Sigma = \{a, b\}$: si

$$\bigcap_{n=1}^{\infty} L_n = \Sigma^* \setminus \{a^n b^n \mid n \geq 1\}$$

était régulier, son complémentaire $\{a^n b^n \mid n \geq 1\}$ le serait aussi.

IV – Reconnaissable $\Rightarrow$ régulier par programmation dynamique

Cet exercice est une alternative à la méthode d'élimination des états pour obtenir une expression régulière à partir d'un automate, avec une méthode similaire à l'algorithme de Floyd-Warshall.

Soit $(\Sigma, Q, 0, F, \delta)$ un automate déterministe tel que $Q = \{0, \dots, n-1\}$.

Soit $L(i, j, k)$ le langage des étiquettes des chemins de i à j n'utilisant que des états intermédiaires strictement inférieurs à k .

1. Montrer que $L(i, j, 0)$ est régulier, pour tous les états i, j .

Soient i et j deux états (éventuellement égaux) et $a_1, \dots, a_p$ toutes les étiquettes des transitions de i vers j . Si $i \neq j$, $L(i, j, 0) = \{a_1, \dots, a_p\}$. Si $i = j$, il faut aussi compter le chemin de longueur nulle : $L(i, i, 0) = \{a_1, \dots, a_p\} \cup \{\varepsilon\}$. Dans les deux cas, le langage est fini donc régulier.

2. Donner une équation de récurrence sur $L(i, j, k)$, en la démontrant.

Pour $0 \leq k < n$, soit $u \in L(i, j, k+1)$ l'étiquette d'un chemin C de i à j n'utilisant que des états intermédiaires inférieurs ou égaux à k . Si C n'utilise pas l'état k comme état intermédiaire, alors $u \in L(i, j, k)$. Sinon, en coupant à chaque passage intermédiaire par k , C est la concaténation d'un chemin de i à k , d'un certain nombre de cycles de k à k , puis d'un chemin de k à j , dont les états intermédiaires sont tous strictement inférieurs à k . Réciproquement, une telle concaténation ne possède que des états intermédiaires inférieurs ou égaux à k . On en déduit :

$$L(i, j, k+1) = L(i, j, k) \cup L(i, k, k)L(k, k, k)^*L(k, j, k).$$

3. En déduire, par récurrence, que tout langage reconnaissable est régulier.

Montrons par récurrence sur $k \in \{0, \dots, n\}$ la propriété $P(k)$: « pour tous états i, j , $L(i, j, k)$ est régulier ». $P(0)$ est vraie d'après la question 1.

Supposons $P(k)$ vraie pour un k tel que $0 \leq k < n$. Les langages $L(i, j, k)$, $L(i, k, k)$, $L(k, k, k)$ et $L(k, j, k)$ sont réguliers par hypothèse de récurrence. La relation de la question précédente montre que $L(i, j, k+1)$ est régulier par stabilité par union, concaténation et étoile. Donc $P(k+1)$ est vraie.

Le langage reconnu est $\bigcup_{f \in F} L(0, f, n)$, une union finie de langages réguliers ; il est donc régulier. Tout langage reconnaissable admet un automate déterministe, ce qui donne le résultat annoncé.

4. Écrire le pseudocode d'un algorithme ayant pour entrée un automate déterministe et renvoyant une expression régulière dénotant le langage reconnu par cet automate. Préciser la complexité.

On manipule des expressions avec les constructeurs Vide, Epsilon, Lettre(a), Union(e, f), Concat(e, f) et Etoile(e). Pour chaque k , la case $R[i, j]$ dénote $L(i, j, k)$.

```

R ← matrice n × n remplie avec Vide
pour i de 0 à n - 1 :
    R[i,i] ← Epsilon
pour chaque transition i → j :
    R[i,j] ← Union(R[i,j], Lettre(a))
pour k de 0 à n - 1 :
    T ← nouvelle matrice n × n
    pour i de 0 à n - 1 :
        pour j de 0 à n - 1 :
            T[i,j] ← Union(R[i,j],
                            Concat(R[i,k], Concat(Etoile(R[k,k]), R[k,j])))
    R ← T
e ← Vide
pour chaque état final f :
```

```

e ← Union(e, R[0,f])
renvoyer e

```

L'initialisation dénote les chemins sans état intermédiaire, y compris le chemin vide sur la diagonale. La récurrence démontrée assure l'invariant après chaque tour. La matrice T évite de mélanger deux valeurs de k . L'union finale dénote donc le langage reconnu, y compris lorsque $F = \emptyset$.

Soit m le nombre de transitions. Il y a n^3 mises à jour. Si chaque constructeur crée un nœud immuable contenant des références vers ses sous-expressions, son coût est constant : le temps et la mémoire totale sont $O(n^3 + m)$; les deux matrices seules occupent $O(n^2)$ cases. Les sous-expressions communes sont partagées, sans être recopiées.

Si l'on veut écrire l'expression entière sous forme de texte, il faut déplier ces références. Une mise à jour peut recopier quatre expressions du tour précédent : la taille peut donc croître exponentiellement en n . Le coût de cette écriture dépend de la taille produite ; il ne se limite pas à $O(n^3)$.

V – Résiduels et minimisation d'automate

Si $u \in \Sigma^*$ et L est un langage sur Σ , on appelle résiduel de L par rapport à u le langage $u^{-1}L$ tel que :

$$u^{-1}L = \{m \in \Sigma^* \mid um \in L\}.$$

Intuitivement, le résiduel de L par rapport à u est l'ensemble des mots avec lesquels on peut compléter u pour obtenir un mot de L . L'ensemble des résiduels de L est noté $\mathcal{R}_L = \{u^{-1}L \mid u \in \Sigma^*\}$.

1. Si L est un langage, quel est le résiduel de L par rapport à ε ?

$\varepsilon^{-1}L = L$. En effet, si $m \in \Sigma^*$, $m \in \varepsilon^{-1}L \Leftrightarrow \varepsilon m = m \in L$.

2. Montrer que pour tout langage L sur Σ et tous mots $u, v \in \Sigma^*$, $u^{-1}(v^{-1}L) = (vu)^{-1}L$.

Soient L un langage et $u, v \in \Sigma^*$. Si $m \in \Sigma^*$, alors :

$$m \in u^{-1}(v^{-1}L) \Leftrightarrow um \in v^{-1}L \Leftrightarrow vum \in L \Leftrightarrow m \in (vu)^{-1}L.$$

3. a) Soit $L_1 = (a|b)^*ab(a|b)^*$. Déterminer $(bab)^{-1}L_1$ et $(aaaba)^{-1}L_1$. Que constate-t-on ?
b) Expliciter $\mathcal{R}_{L_1}$.

a) Le langage L_1 est l'ensemble des mots contenant ab . Les deux mots proposés contiennent déjà le motif ab : on peut donc les compléter par n'importe quel mot sur $\{a, b\}$ et rester dans L_1 . On en déduit $(bab)^{-1}L_1 = (aaaba)^{-1}L_1 = (a|b)^*$.

b) On a

$$u^{-1}L_1 = \begin{cases} (a|b)^* & \text{si } u \in L_1 \\ L_1 \cup b(a|b)^* & \text{si } u \notin L_1 \text{ et finit par un } a \\ L_1 & \text{si } u = \varepsilon \text{ ou si } u \notin L_1 \text{ et finit par un } b. \end{cases}$$

D'où $\mathcal{R}_{L_1} = \{(a|b)^*, L_1 \cup b(a|b)^*, L_1\}$.

On veut montrer qu'un langage L est reconnaissable si et seulement s'il admet un nombre fini de résiduels.

4. Soit L un langage reconnaissable par un automate $A = (\Sigma, Q, q_0, F, \delta)$ qu'on peut supposer déterministe, complet et dont tous les états sont accessibles sans perte de généralité.

- a) Montrer que la fonction

$$\begin{aligned} \varphi : Q &\rightarrow \mathcal{R}_L \\ q &\mapsto u^{-1}L \quad \text{où } \delta^*(q_0, u) = q \end{aligned}$$

est correctement définie : u est l'un des mots qui permettent d'atteindre q depuis q_0 .

- b) Montrer que L admet un nombre fini de résiduels.

a) Soit $q \in Q$. Comme A est accessible, il existe un mot u permettant d'atteindre q depuis q_0 . Le déterminisme de A assure que $\delta^*(q_0, u)$ est un seul état : q . Il existe donc bien un mot u tel que $\delta^*(q_0, u) = q$, ce qui permet de construire l'image de q par φ .

Il reste à montrer que cette image ne dépend pas du choix de u . Considérons deux mots u, v tels que $\delta^*(q_0, u) = \delta^*(q_0, v)$. Pour tout $m \in \Sigma^*$,

$$\begin{aligned} m \in u^{-1}L &\Leftrightarrow um \in L \\ &\Leftrightarrow \delta^*(q_0, um) \in F \\ &\Leftrightarrow \delta^*(\delta^*(q_0, u), m) \in F \\ &\Leftrightarrow \delta^*(\delta^*(q_0, v), m) \in F \\ &\Leftrightarrow \delta^*(q_0, vm) \in F \\ &\Leftrightarrow vm \in L \Leftrightarrow m \in v^{-1}L. \end{aligned}$$

Ainsi $u^{-1}L = v^{-1}L$: le résiduel associé à q par φ est indépendant du choix de u .

- b) Montrons que φ est surjective. Soit $u^{-1}L$ un résiduel de L . Comme A est complet, le mot u peut être lu à partir de q_0 : il existe $q \in Q$ tel que $\delta^*(q_0, u) = q$. En particulier, $u^{-1}L = \varphi(q)$.

Ceci garantit que $|Q| \geq |\mathcal{R}_L|$. L'automate A étant fini, le langage L a un nombre fini de résiduels.

5. Soit à présent L un langage ayant un nombre fini de résiduels. On définit alors un automate $M(L) = (\Sigma, Q, I, F, \delta)$, appelé automate des résiduels associé à L , où :

- $Q = \mathcal{R}_L$;
 - $I = \{\varepsilon^{-1}L\}$;
 - $F = \{u^{-1}L \mid u \in L\}$;
 - pour toute lettre $a \in \Sigma$ et tout résiduel $u^{-1}L \in Q$, $\delta(u^{-1}L, a) = (ua)^{-1}L$.
- a) Montrer que la fonction δ est correctement définie, c'est-à-dire que l'image donnée par δ d'un résiduel $u^{-1}L$ ne dépend que du résiduel et pas de u .
- b) Montrer que $M(L)$ est un automate déterministe et complet.
- c) Montrer que $M(L)$ reconnaît le langage L .

- a) Soient $u, v \in \Sigma^*$ tels que $u^{-1}L = v^{-1}L$, et $a \in \Sigma$. Pour tout $m \in \Sigma^*$, d'après la définition et la question 2,

$$\begin{aligned} m \in \delta(u^{-1}L, a) &\Leftrightarrow m \in (ua)^{-1}L \\ &\Leftrightarrow uam \in L \\ &\Leftrightarrow am \in u^{-1}L \\ &\Leftrightarrow am \in v^{-1}L \\ &\Leftrightarrow m \in a^{-1}(v^{-1}L) = \delta(v^{-1}L, a). \end{aligned}$$

Ces deux langages sont donc égaux. L'état atteint en lisant a depuis le résiduel $u^{-1}L$ ne dépend que de ce résiduel, et non du choix de u .

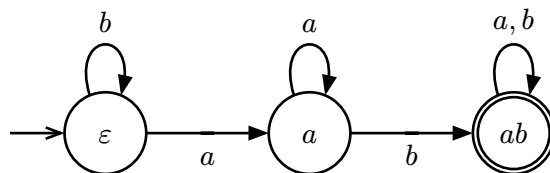
- b) Par hypothèse, l'ensemble des résiduels de L , qui est aussi l'ensemble des états de $M(L)$, est fini. Donc $M(L)$ est un automate fini. Il est déterministe et complet par construction de sa fonction de transition et parce qu'il a un seul état initial.
- c) Procédons par double inclusion.

Si $m \in L$, $\delta^*(\varepsilon^{-1}L, m) = m^{-1}L$ en lisant le mot m lettre à lettre et en utilisant la question 2. Or $m^{-1}L \in F$ puisque $m \in L$. Lire m depuis l'état initial conduit donc à un état final : m est reconnu par $M(L)$.

Réciproquement, si m est reconnu par $M(L)$, alors il existe $u \in L$ tel que $m^{-1}L = \delta^*(\varepsilon^{-1}L, m) = u^{-1}L$. Comme $u \in L$, $\varepsilon \in u^{-1}L$; on a donc aussi $\varepsilon \in m^{-1}L$, ce qui garantit que $m \in L$.

6. À l'aide des questions précédentes, déterminer l'automate des résiduels associé au langage L_1 .

Les états sont étiquetés par les représentants ε, a, ab de leurs résiduels, respectivement $L_1, L_1 \cup b(a|b)^*$ et $(a|b)^*$.



Un automate reconnaissant un langage L est dit minimal s'il est déterministe, complet et possède le plus petit nombre d'états possible parmi les automates déterministes et complets reconnaissant L .

7. Montrer que l'automate des résiduels est un automate minimal.

D'après la question 5, $M(L)$ est déterministe, complet et reconnaît L . Il possède exactement $|\mathcal{R}_L|$ états. Soit A un autre automate déterministe complet reconnaissant L . On peut supprimer ses états inaccessibles : les transitions issues d'un état accessible aboutissent à des états accessibles, donc l'automate obtenu reste complet et reconnaît L . La surjection construite à la question 4, de ses états accessibles vers $\mathcal{R}_L$, montre qu'il possède au moins $|\mathcal{R}_L|$ états. A fortiori, A possède au moins autant d'états que $M(L)$. Ainsi $M(L)$ atteint le plus petit nombre d'états possible : il est minimal.

On peut montrer que cet automate minimal est en fait unique (à renommage des états près), ce qui fournit une façon de déterminer si deux automates reconnaissent le même langage : il suffit de les minimiser et de comparer les deux automates obtenus (une autre méthode consistant à tester si $L_1 \Delta L_2 = \emptyset$).

Il est possible de déterminer l'automate minimal équivalent à un automate donné grâce à l'algorithme de Moore.